Implementation

Frontend Overview

The SafeRide School mobile application was developed using Flutter, Google's open-source UI framework, which uses Dart as the programming language. Flutter was chosen for its ability to produce a single codebase that runs natively on both Android and iOS, offering a consistent user experience across devices.

Architecture

The frontend follows the MVC (Model-View-Controller) pattern. Models define the data structures for entities such as users, students, buses, trips and notifications. Views are the Flutter widget-based screens that present information to each user role. Controllers are implemented as Provider ChangeNotifier classes that manage state and sit between the data layer and the UI, ensuring screens rebuild only when relevant data changes.

Tools and Environment

Tool	Purpose
VS Code	Primary development environment
Pixel 4 (android-x64 emulator)	Android build and device emulation
GitHub	Version control and collaboration
Figma	UI mockups and design prototyping

Libraries and Packages

Packages such as provider, file_picker, csv, and http were used to manage state across the widget tree, select CSV files from device storage, parse uploaded CSV files into structured Dart objects, and handle REST API communication with the backend, respectively.

Key Components Built

Screens: Welcome, Sign Up, Login, and role-specific dashboards for Parents, Drivers, and School Administrators.

Role-based dashboard: A single DashboardScreen entry point reads the authenticated user's role and renders the appropriate view, giving each role access only to their relevant features and data.

CSV upload flow: Administrators can upload a student list as a CSV file, preview parsed entries with inline validation, and confirm the upload, which posts the data to the backend API.

Driver attendance: Drivers can mark each student as present or absent directly from their dashboard using inline toggle controls.

Navigation: All screens are connected with role-aware routing.

Backend Overview

The SafeRide School backend was developed using NestJS with TypeScript and PostgreSQL as the primary relational database. NestJS was chosen because it provides a modular and scalable structure for building REST APIs, while PostgreSQL offers reliable storage for structured application data such as users, students, buses, trips, attendance records, and notifications.

The backend is responsible for handling business logic, securing application access, validating requests, managing persistent data, and serving role-specific information to the Flutter frontend. It exposes RESTful API endpoints that allow different users, such as school administrators, drivers, and parents, to interact with the system based on their assigned permissions.

Architecture

The backend follows a modular architecture built around NestJS modules, controllers, and services. Each major feature of the system is separated into its own domain area, improving maintainability and reducing coupling between components.

Controllers define the API endpoints and handle incoming HTTP requests. Services contain the main business logic, such as processing attendance updates, validating CSV uploads, or retrieving dashboard data. Database operations are handled through the persistence layer, which communicates with PostgreSQL to create, read, update, and delete records.

This separation ensures that presentation concerns, request handling, business rules, and database operations are kept distinct. It also makes the system easier to test, extend, and debug.

Authentication and Authorization

Authentication in the SafeRide School backend was implemented using Better Auth. Better Auth was used to manage secure user authentication flows, session handling, and identity verification. Rather than building authentication entirely from scratch, it provided a structured and secure approach for handling login and user identity management.

When a user signs up or logs in, Better Auth verifies their credentials and establishes an authenticated session. The backend then uses this authenticated context to determine who the user is and what role they belong to. This is critical in a system like SafeRide School because different types of users need access to different parts of the platform.

For example, a school administrator can upload student data, manage transport records, and oversee operational activity. A driver can access trip and attendance-related functions but should not be able to manage administrative records. A parent can view information relevant to their child but should not be able to modify school or transport data. Authentication confirms the user's identity, while authorization ensures that the user only accesses resources allowed for their role.

To enforce this, the backend applies role-based access control on protected routes. After a user is authenticated, guards and authorization checks ensure that only users with the correct role can access specific endpoints. This prevents unauthorized access to sensitive operations and helps maintain data security across the system.

In addition, Better Auth improves the security model of the application by centralizing authentication logic. This reduces the risk of inconsistent authentication handling across modules and makes it easier to manage sessions, protect routes, and maintain secure login behavior.

Database Design

PostgreSQL was used as the backend database because the application data is highly relational. The system includes multiple connected entities such as users, students, buses, trips, attendance entries, and notifications. A relational database was appropriate because it supports structured schemas, foreign key relationships, and data consistency.

For example, users are linked to roles, students may be linked to parent accounts, drivers may be linked to assigned buses or trips, and attendance records are linked to both students and specific trips. This design allows the system to retrieve accurate, role-specific information efficiently while maintaining referential integrity.

Core Backend Features

The backend supports several key operations required by the SafeRide School platform. These include user authentication, secure access to protected routes, role-specific dashboard data retrieval, student management, trip and bus record handling, attendance updates, notification storage or retrieval, and CSV-based student import.

A particularly important feature is the CSV upload process for school administrators. When a CSV file is submitted from the frontend, the backend receives the file, parses its content, validates each row against the required format, checks for missing or invalid values, and converts valid entries into structured records before saving them to PostgreSQL. This allows schools to upload large sets of student data efficiently while reducing manual input errors.

The driver attendance feature is also handled by the backend. When a driver marks students as present or absent, the backend validates the request, updates the corresponding attendance records in the database, and ensures that the action is only available to an authenticated user with the correct driver role.

Validation and Security

The backend includes validation mechanisms to ensure that incoming data is well-formed and safe before it reaches the database. Request validation is applied to important operations such as registration, login, CSV processing, and attendance updates. This helps prevent invalid records, inconsistent data, and misuse of endpoints.

Security is further strengthened through authenticated route protection, role-based authorization, and structured request handling. Since the application handles

school-related and student-related records, these protections are essential to prevent unauthorized access and maintain trust in the system.

Tools and Environment

The backend was developed using NestJS, TypeScript, PostgreSQL, Visual Studio Code, and GitHub. Better Auth was integrated to provide secure authentication and session management. The backend communicates with the Flutter frontend through HTTP-based API calls and serves as the central layer for data processing, access control, and persistent storage.